Build Task — Project 3: LangGraph App — Hints & Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

This Build Task **is** Project 3 — the full step-by-step build lives in Project 3's READING_README. The solution below is the same graph at its smallest, in one folder, so you can see every piece working together first.

- Hint 1 — The shape, and the exact pieces
- Hint 2 — The plan, and almost the whole thing
- Solution

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 1 — The shape, and the exact pieces

On top of that:

- **A visible path:** a `path: Annotated[list[str], operator.add]` field; every node adds one short entry (`"searched"` , `"no grounding"` , `"approved"`), so the final state says exactly what happened.
- **`build_graph(checkpointer=None)`** — Doc09's Build Task, so a test can pass its own.
- **`test_project3.py`** — the README's 5 Test Cases, each printing `OK` or `WRONG` .
- **Retry on search** — the Failure exercise's `search_with_retry()` .
- **Who approved, and when** — the Real-world exercise's structured resume payload.
- **A validated state** — Pydantic's `BaseModel` (Doc04) instead of `TypedDict` . Watch out: nodes then get an **object**, so they read `state.task` , not `state["task"]` .

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
state.py:   + path: Annotated[list[str], operator.add]
nodes.py:   each node returns its path entry too, e.g. {"path": ["searched"]}
graph.py:   build_graph(checkpointer=None)
test_project3.py:
    run(graph, task, thread_id, decision) -> final state values
    case 1 search + found + approved   case 2 no search
    case 3 nothing relevant            case 4 approved -> answer == draft
    case 5 rejected -> "Not approved"

Approach 2: search_node calls search_with_retry(); the resume value is
            {"decision": ..., "by": ..., "at": ...}
Approach 3: ProjectState(BaseModel) with defaults; nodes use state.task
```

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Solution

Story — `state.py` / `nodes.py` / `test_project3.py` : Basic works, but "what happened?" means scrolling through step lines, and the 5 Test Cases are checked by hand. **If not:** a change that broke the rejection path would only be noticed when a rejected answer went out anyway.

```
# state.py
import operator
from typing import Annotated, TypedDict

class ProjectState(TypedDict):
    task: str
    sources: list[dict]
    draft: str
    approved: bool
    answer: str
    # every node adds its own name, so the path taken is visible
    path: Annotated[list[str], operator.add]
```

```
# nodes.py
from langchain_openai import ChatOpenAI
from langgraph.types import interrupt

from kb import collection
from retriever import retrieve
from state import ProjectState
```

```
KEYWORDS = ["coffee", "brew", "grind", "water", "beans", "according to"]
MAX_DISTANCE = 1.2

def route_search(state: ProjectState) -> str:
    task = state["task"].lower()
    for keyword in KEYWORDS:
        if keyword in task:
            return "search"
    return "skip"

def search_node(state: ProjectState) -> dict:
    sources = retrieve(collection, state["task"])
    return {"sources": sources, "path": ["searched"]}

def best_distance(sources: list[dict]) -> float:
    best = sources[0]["score"]
    for source in sources:
        if source["score"] < best:
            best = source["score"]
    return best

def has_grounding(state: ProjectState) -> str:
    if not state["sources"]:
        return "no_grounding"
    if best_distance(state["sources"]) > MAX_DISTANCE:
        return "no_grounding"
    return "reason"

def reason_node(state: ProjectState) -> dict:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    if not state.get("sources"):
        draft = model.invoke(state["task"]).content
        return {"draft": draft, "path": ["answered directly"]}
    pieces: list[str] = []
    for source in state["sources"]:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = f"Context:\n{context}\n\nQuestion: {state['task']}"
    draft = model.invoke(prompt).content
    return {"draft": draft, "path": ["answered from documents"]}

def no_grounding_node(state: ProjectState) -> dict:
    return {
        "answer": "I don't have grounding for this in my documents.",
        "path": ["no grounding"],
    }
```

```
def approval_node(state: ProjectState) -> dict:
    decision = interrupt({
        "draft": state["draft"],
        "prompt": "Approve this answer before it's sent? (approved/rejected)",
    })
    approved = decision == "approved"
    if approved:
        return {"approved": True, "path": ["approved"]}
    return {"approved": False, "path": ["rejected"]}

def finish_node(state: ProjectState) -> dict:
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- nothing was sent."}
```

```
# graph.py
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, StateGraph

from nodes import (
    approval_node,
    finish_node,
    has_grounding,
    no_grounding_node,
    reason_node,
    route_search,
    search_node,
)
from state import ProjectState

def build_graph(checkpointer=None):
    if checkpointer is None:
        checkpointer = MemorySaver()
    builder = StateGraph(ProjectState)
    builder.add_node("search_node", search_node)
    builder.add_node("reason_node", reason_node)
    builder.add_node("no_grounding_node", no_grounding_node)
    builder.add_node("approval_node", approval_node)
    builder.add_node("finish_node", finish_node)

    builder.set_conditional_entry_point(
        route_search, {"search": "search_node", "skip": "reason_node"}
    )
    builder.add_conditional_edges(
        "search_node", has_grounding,
        {"reason": "reason_node", "no_grounding": "no_grounding_node"},
```

```
    )
    builder.add_edge("reason_node", "approval_node")
    builder.add_edge("approval_node", "finish_node")
    builder.add_edge("no_grounding_node", END)
    builder.add_edge("finish_node", END)
    return builder.compile(checkpointer=checkpointer)
```

```
# main.py
from langgraph.types import Command

from graph import build_graph

def show_steps(graph, graph_input, config: dict) -> None:
    for update in graph.stream(graph_input, config, stream_mode="updates"):
        for node_name in update:
            print(f"  step: {node_name}")

def ask_reviewer(draft: str) -> str:
    print(f"Draft: {draft}")
    answer = input("Approve? (yes/no): ").strip().lower()
    if answer == "yes":
        return "approved"
    return "rejected"

def main() -> None:
    graph = build_graph()
    config = {"configurable": {"thread_id": "run-1"}}
    task = input("Your question: ")
    show_steps(graph, {"task": task, "path": []}, config)

    paused = graph.get_state(config)
    if paused.next:
        decision = ask_reviewer(paused.values["draft"])
        show_steps(graph, Command(resume=decision), config)

    final = graph.get_state(config).values
    print(f"Path:   {final['path']}")
    print(f"Answer: {final['answer']}")

if __name__ == "__main__":
    main()
```

```
# test_project3.py
# Runs the 5 Test Cases from the README. Each case prints OK or WRONG.
# Run it from inside this folder: python test_project3.py
```

```
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
from langgraph.types import Command

from graph import build_graph

def run(graph, task: str, thread_id: str, decision: str | None) -> dict:
    config = {"configurable": {"thread_id": thread_id}}
    graph.invoke({"task": task, "path": []}, config)
    if decision is not None and graph.get_state(config).next:
        graph.invoke(Command(resume=decision), config)
    return graph.get_state(config).values

def check(case: str, passed: bool, path: list[str]) -> None:
    result = "OK"
    if not passed:
        result = "WRONG"
    print(f"{case} -> {result}: {path}")

if __name__ == "__main__":
    graph = build_graph()
    found = run(graph, "How hot should the water be?", "t1", "approved")
    check("case 1 (search, found)   ",
          found["path"][0] == "searched" and "approved" in found["path"],
          found["path"])

    direct = run(graph, "What is 2 + 2?", "t2", "approved")
    check("case 2 (no search)       ",
          "searched" not in direct["path"], direct["path"])

    nothing = run(graph, "Which coffee suits a marathon?", "t3", None)
    check("case 3 (nothing relevant)",
          nothing["path"] == ["searched", "no grounding"], nothing["path"])

    check("case 4 (approved)        ",
          found["answer"] == found["draft"], found["path"])

    rejected = run(graph, "How hot should the water be?", "t5", "rejected")
    check("case 5 (rejected)        ",
          rejected["answer"].startswith("Not approved"), rejected["path"])
```

Expected output of `python test_project3.py`:

```
case 1 (search, found)    -> OK: ['searched', 'answered from documents',
'approved']
case 2 (no search)        -> OK: ['answered directly', 'approved']
case 3 (nothing relevant) -> OK: ['searched', 'no grounding']
```

```
case 4 (approved)         -> OK: ['searched', 'answered from documents',
'approved']
case 5 (rejected)         -> OK: ['searched', 'answered from documents',
'rejected']
```

(Wrapped to fit the page.) Each node returns its own one-item `path` list, and `operator.add` joins them in order — Doc09's reducer.